



Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Curso: CE-1101 Introducción a la programación

Profesor: Ellioth Alberto Ramírez Trejos

DOCUMENTO TECNICO

Proyecto: Imaginary Battles

Autor: Luis Diego Murillo Matarrita

Fecha: 8/5/ 2026

Versión: 1.0

Índice

Palabras clave y Acrónimos	3
Introducción	4
Marco Teórico	5
Descripción del Programa	6
Diagrama de flujo general del sistema	6
Resultados Obtenidos	7
Descripción de las Funciones Principales	7
Descripción de la Interfaz Gráfica	8
Conclusiones y Recomendaciones	9
Conclusiones	9
Recomendaciones	9
Bibliografía	10

Palabras clave y Acrónimos

Palabras clave: juego por turnos, interfaz gráfica, recursividad, Python, Tkinter, personajes, batalla, listbox, radiobuttons

Acrónimos:

- GUI: Graphical User Interface (Interfaz Gráfica de Usuario)
- KO: Knock Out (personaje sin puntos de vida)
- ATK: Attack (puntos de ataque del personaje)
- DEF: Defense (puntos de defensa del personaje)
- HP: Hit Points (puntos de vida del personaje)
- CSV: Comma Separated Values (valores separados por comas)

Introducción

El presente documento técnico describe el desarrollo del proyecto Imaginary Battles, un juego de batalla por turnos desarrollado en el lenguaje de programación Python.

El juego sigue la temática de un Guardian de las Historias que debe recuperar personajes capturados por Los Huecos (Hollows), seres que desean borrar las historias mas populares. El jugador debe recorrer 5 tierras diferentes y vencer a los Hollows en combates por turnos para restaurar la narrativa original.

El proyecto fue desarrollado individualmente, utilizando Tkinter como librería de interfaz gráfica, PIL/Pillow para el manejo de imágenes, uso de IA para consultas y recomendaciones, y GitHub para el control de versiones. Se aplico recursividad en las funciones principales de lógica del juego como lo exigen los requerimientos del proyecto.

Marco Teórico

El sistema fue desarrollado utilizando el lenguaje de programación Python, seleccionado por su facilidad de aprendizaje, legibilidad y amplio ecosistema de bibliotecas. El enfoque de desarrollo se basa en principios de programación modular, separando la lógica del juego de la interfaz gráfica.

Tecnologías empleadas

- Python 3.13: Lenguaje principal de desarrollo
- Tkinter: Librería estándar de Python para interfaces graficas de usuario
- PIL / Pillow: Librería para carga y manipulación de imágenes
- Random: Modulo estándar para generación de valores aleatorios
- Git / GitHub: Control de versiones y repositorio público del proyecto
- IA: Inteligencia Artificial

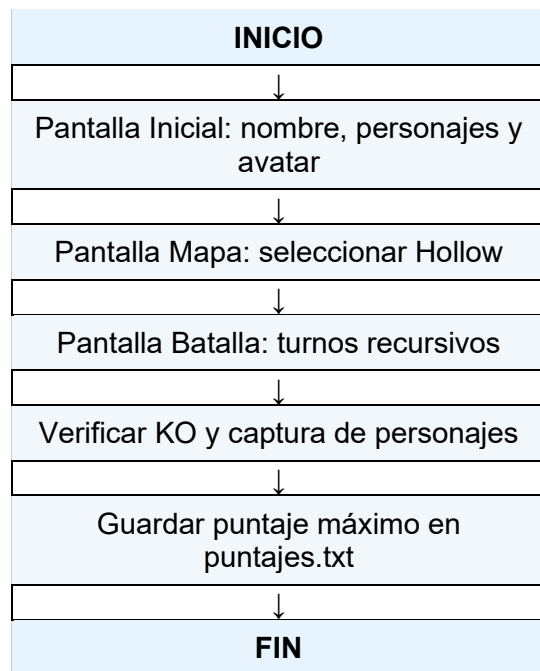
Conceptos aplicados

- Recursividad: Función que se llama a si misma con un caso base y un caso recursivo
- Programación orientada a objetos: Uso de clases y herencia para las pantallas de Tkinter
- Estructuras de datos: Uso de listas y diccionarios para representar personajes y hollows
- Manejo de archivos: Lectura de personajes desde CSV y escritura del puntaje máximo

Descripción del Programa

El programa inicia con la carga de los 15 personajes desde el archivo Personajes.txt. El jugador ingresa su nombre, selecciona 3 personajes y un avatar en la pantalla inicial. Luego navega por el mapa eligiendo cuales Hollows enfrentar en orden. Cada batalla se desarrolla por turnos hasta que uno de los dos jugadores se queda sin personajes. Al finalizar todas las batallas se guarda el puntaje máximo.

Diagrama de flujo general del sistema



Resultados Obtenidos

Descripción de las Funciones Principales

A continuación, se describen las funciones principales del sistema, sus entradas, salidas y restricciones:

Función	Entradas	Salidas	Restricciones	Funcionamiento
load_Personajes()	Archivo Personajes.txt	Lista de diccionarios con los 15 personajes	El archivo debe existir y tener formato CSV correcto	Lee el archivo línea por línea y construye un diccionario por personaje
dañoRecibido(atk, def)	ATK del atacante, DEF del defensor (enteros)	Entero con el daño calculado (mínimo 1)	Ambos valores deben ser enteros positivos	Función recursiva: reduce ATK y DEF en 1 por llamada hasta que DEF llegue a 0
turnos(turno, p_jugador, p_hollow, log)	Numero de turno, personaje jugador, personaje hollow, lista log	Log actualizado con los resultados del turno	Los personajes no deben ser None	Recursiva: turno par=jugador ataca, impar=hollow ataca. Para cuando alguien queda en KO
Crearhollows(personajes)	Lista de 15 personajes	Lista de 5 hollows con 3 personajes cada uno	La lista debe tener exactamente 15 personajes	Mezcla los personajes aleatoriamente y los reparte entre 5 hollows
guardarPuntaje(nombre, puntaje)	Nombre del jugador, puntaje obtenido	Archivo puntajes.txt actualizado	Solo guarda si el puntaje es mayor al existente	Lee el puntaje guardado y lo reemplaza si el nuevo es mayor

Descripción de la Interfaz Grafica

La interfaz gráfica del sistema fue desarrollada con Tkinter y consta de tres pantallas principales:

- Pantalla Inicial: Permite al jugador ingresar su nombre, seleccionar 3 personajes de una lista de 15 usando un Listbox con selección múltiple, elegir uno de 3 avatares con Radiobuttons, e iniciar el juego. Incluye un boton ABOUT con información del proyecto.
- Pantalla de Mapa: Muestra las 5 tierras con sus respectivos Hollows como botones. Solo el primer Hollow está disponible al inicio; los demás se desbloquean progresivamente al vencer al anterior. Los Hollows vencidos aparecen deshabilitados.
- Pantalla de Batalla: Muestra el nombre del jugador, su avatar, los personajes en combate con sus HP actuales e imágenes, un log de batalla con los mensajes de cada turno, el puntaje actual y botones para ATACAR o cambiar de personaje. Al cambiar de personaje se abre una ventana emergente con los personajes disponibles.

Conclusiones y Recomendaciones

Conclusiones

- Se logro implementar un juego de batalla por turnos funcional en Python usando Tkinter como interfaz gráfica, cumpliendo con todos los requerimientos del proyecto.
- La recursividad fue aplicada de forma natural en dos funciones clave: el cálculo del daño (dañoRecibido) y la simulación de turnos (turnos), lo que permitió comprender su uso practico.
- La separación del código en módulos (logica.py, pantalla.py, mapa.py, batalla.py, boceto.py) facilito el desarrollo del proyecto.
- El uso de GitHub permitió mantener un historial de cambios organizado y garantizar la persistencia del código.
- La integración de imágenes con PIL/Pillow enriqueció visualmente la experiencia del juego sin complicar significativamente el código.

Recomendaciones

- Se recomienda agregar música y efectos de sonido para mejorar la experiencia del jugador usando la librería pygame o similar en futuras versiones.
- Implementar un sistema de niveles o dificultad variable para los Hollows, haciendo el juego más retador conforme avanza el jugador.
- Agregar animaciones simples en los ataques para hacer la batalla más dinámica visualmente.
- Se recomienda implementar un sistema de guardado completo del estado del juego para que el jugador pueda continuar una partida en otro momento.
- El tener separado por módulos el código para mantener un orden en el desarrollo del proyecto

Bibliografía

- Python Software Foundation. (2024). *Python documentation*. <https://docs.python.org>
- Python Software Foundation. (2024). *tkinter — Python interface to Tcl/Tk*. <https://docs.python.org/3/library/tkinter.html>
- Clark, A. (2024). *Pillow (PIL Fork) documentation*. <https://pillow.readthedocs.io>
- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book>
- GitHub, Inc. (2024). *GitHub documentation*. <https://docs.github.com>